

Specyfikacja Projektu Zaliczeniowego

Michał Gremplewicz, Mateusz Bosak

1. Tytuł Projektu

System egzaminów online z licznikiem czasu i automatycznym przesłaniem

2. Opis i cele projektu

Projekt polega na stworzeniu aplikacji webowej umożliwiającej przeprowadzanie egzaminów online z ograniczeniem czasowym. System automatycznie zapisuje odpowiedzi użytkownika oraz kończy egzamin po upływie wyznaczonego czasu.

Aplikacja posiada dwa główne moduły:

- Student - rozwiązuje testy
- Nauczyciel - zarządza pytaniami i ma wgląd do odpowiedzi

Cele projektu:

- Automatyzacja egzaminów
- Kontrola czasu rozwiązywania testu
- Losowanie pytań
- Automatyczna ocena wyników
- Przechowywanie danych w bazie

3. Wymagania Funkcjonalne

Student:

- Logowanie do panelu studenta
- Rozpoczęcie testu z licznikiem czasu
- Odpowiadania na pytania
- Automatyczne zakończenie egzaminu po upływie czasu
- Podgląd wyników

Administrator:

- Logowanie do panelu Administrator
- Utworzenie testu z bazy pytań i ustawienie czasu
- Podgląd wyników

4. Wymagania Niefunkcjonalne

- Niezawodność - pytania zapisują się w przypadku utraty połączenia
- Wydajność - szybka odpowiedź serwera w granicy 2 sekund
- Bezpieczeństwo - użytkownik loguje się przy użyciu loginu i hasła
- Zgodność - responsywny interfejs
- Skalowalność - obsługa co najmniej 2 użytkowników naraz

5. Architektura Systemu

Typ architektury: Klient-Serwer

Schemat:

- Frontend: React.js + TailwindCSS
- Backend: Java
- Baza danych: PostgreSQL
- WebSocket: WebSocket
- Autoryzacja: JWT

6. Zastosowane algorytmy

- Losowanie pytań: shuffle listy pytań
- Sprawdzanie odpowiedzi: porównywanie odpowiedzi z kluczem
- Zegar odliczający czas do zakończenia testu: Timestamp + TTL
- Filtrowanie danych: SQL
- Autoryzacja: Tokenizacja i walidacja JWT

7. Struktura klas

Model:

- User - id, username, password, role
- Question - id, content, options, correct, points
- Exam - id, title, duration_minutes, questions_list
- Attempt - id, user_id, exam_id, score, start_time, submitted_at

Serwisy:

- GradingService - Obliczanie punktów po zakończeniu testu
- ExamService - Logika losowania pytań, walidacja czasu trwania egzaminu
- AuthService - JWT, walidacja
- UserService - rejestracja, logowanie
- TimerService - obsługa automatycznego przesyłania po terminie.

Kontrolery:

- AuthController - logowanie i uwierzytelnianie
- ExamController - obsługa startu egzaminu i przesyłania wyników
- UserController - REST API użytkownika
- QuestionController - REST API pytań

Dodatkowe komponenty:

- DTO - obiekty wymiany danych
- Repository - interfejsy do komunikacji z bazą danych
- JwtFilter - komponent weryfikujący tożsamość użytkownika przy każdym zapytaniu do API

8. Technologie i biblioteki

Backend:

- Java
- WebSocket

Frontend:

- [React.js](https://reactjs.org/)
- TailwindCSS

Baza danych:

- PostgreSQL

Narzędzia:

- Docker
- Git
-

9. API i Modele Danych

Przykładowe Endpointy:

- POST /api/auth/login
- GET /api/exams
- POST /api/exams
- GET /api/questions
- POST /api/results

Przykładowy model:

```
{  
  "id": "1",  
  "question": "2+2=?",  
  "answers": ["2", "3", "4"],  
  "correct": "4"  
}
```

10. Interfejs użytkownika

Główne widoki:

- Ekran logowania - formularz z walidacją danych
- Lista egzaminów - przegląd dostępnych egzaminów do wykonania
- Ekran testu (timer + pytania)
- Przegląd wyników - panel pozwalający na przejrzanie odpowiedzi
- Panel administratora - umożliwia tworzenie egzaminów i ustawianie czasu

Elementy UX/UI:

- Czytelny interfejs
- Responsywność - dostępność na urządzeniach mobilnych
- Czytelny timer
- Tryb jasny i ciemny

Technologie UI:

- React
- TailwindCSS

11. Etapy Realizacji:

Faza	Opis	Czas
Planowanie	Określenie wymagań, wybór technologii	2 tygodnie
Backend	Budowa API, modele danych	3 tygodnie
Frontend	Interfejs, integracja z API	2 tygodnie
Testy	Testy jednostkowe i integracyjne	1 tydzień
Wdrożenie	Konfiguracja Docker, CI/CD, serwer	1 tydzień

12. Testowanie:

- Testy jednostkowe
- Test API
- Testy E2E - Cypress
- Testy obciążeniowe - JMeter

13. Ryzyka i środki zaradcze

Ryzyko	Wpływ	Środki zaradcze
Błędy w logice sprawdzania	Wysoki	Testy jednostkowe
Problemy z timerem	Średni	Timer po stronie serwera
Utrata połączenia internetowego	Średni	Autosave, zapisy lokalne
Kompatybilność przeglądarek	Niski	Test na różnych przeglądarkach

14. Kryteria sukcesu

- Aplikacja działa bez błędów
- Wszystkie funkcje działają
- Poprawnie zapisywane są wyniki
- Poprawnie odczytuje się wyniki

15. Szacunkowy Czas Pracy

Nasz całkowity szacowany czas pracy wynosi 175 godzin roboczych.

Moduł	Opis	Szacunkowy Czas
Projektowanie	Wymagania, wybór technologii	10
Backend	Logika i modele danych	35
Frontend	Interakcje z API	35
Logowanie i sesje	Proces logowania	15
WebSocket	Live Updates	15
Testy	Pokrycie testami kluczowych ścieżek	15
Docker	Konfiguracja środowiska	10
Dokumentacja techniczna	Komentarze, API scheme	10
Błędy	Debugowanie, nieprzewidziane problemy	30